

Part 2: Jets at the LHC

What a Jet Is, and How We Find One

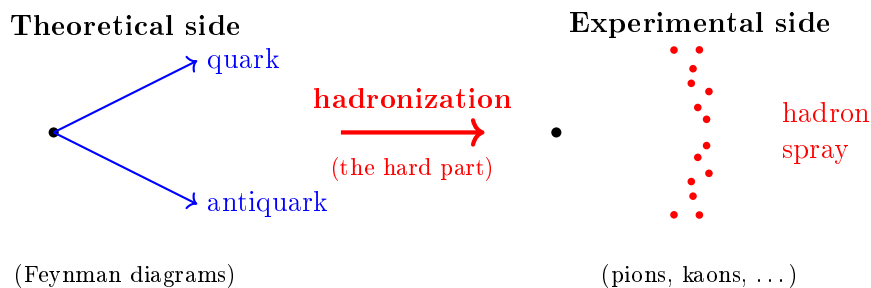
PHYS 7502 – Final Reading Assignment

1 The Core Problem: Partons Versus Hadrons

Let's start with the fundamental mismatch that makes jet physics both necessary and interesting.

On one side, we have the **theory**: QCD is written in terms of quarks and gluons. A Feynman diagram for, say, $pp \rightarrow$ two jets is a diagram for $qq \rightarrow qq$ (or $qg \rightarrow qg$, etc.) at the parton level. You can compute its matrix element with Casimir's trick, convolve with PDFs, and get a prediction for a cross section.

On the other side, we have the **experiment**: what the detector sees are *hadrons* – pions, kaons, protons, photons from neutral pion decays, and occasional neutrinos that escape. Quarks and gluons are colored and cannot propagate as free particles; they confine into hadrons before ever reaching the silicon trackers and calorimeters.



The job of **jet physics** is to build a bridge between these two sides. That bridge has two parts:

1. an *experimental* procedure for grouping detected hadrons into objects called **jets**, and
2. a *theoretical* understanding of why those jets faithfully represent the underlying partons.

Why This Is Nontrivial

You might think: “just draw a cone around each parton direction and call that a jet.” But we don’t know the parton directions – they’re not what’s measured. We see hundreds of hadrons. Which ones belong to which jet? Even harder: is a soft gluon emitted at wide angle part of the jet, or noise? The answers depend on the algorithm you use, and the algorithm *must* be chosen to preserve the theory-experiment correspondence. Get this wrong and your cross section prediction is worthless.

2 What Does a Jet Actually Look Like?

Let’s get concrete. Figure 1 (below) shows the anatomy of a jet, drawn schematically, at progressively shorter time scales.

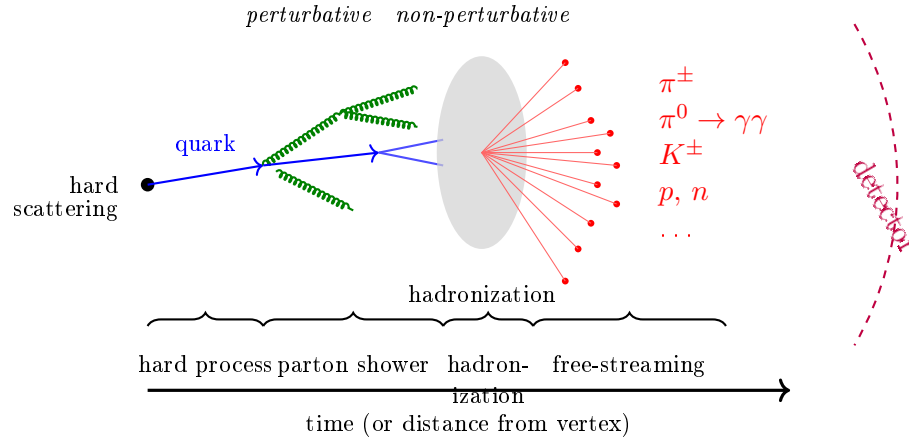


Figure 1: Anatomy of a jet. Starting from a hard-scattered parton (the bare quark), perturbative QCD describes the emission of additional gluons and quark-antiquark pairs (the *parton shower*). At the scale where $\alpha_s \sim 1$, all the colored partons non-perturbatively rearrange into color-singlet *hadrons* (the blob region). These hadrons then free-stream to the detector.

2.1 Two kinds of jets: truth and detector

In practice, when people say “jet”, they could mean one of two things depending on context:

Two Definitions of a Jet

Truth-level (or particle-level) jet. Built from the *final-state stable particles* produced by the event – pions, kaons, protons, etc. This is what you get out of a Monte Carlo simulation before any detector effects. It’s the “theorist’s jet”.

Detector-level jet. Built from whatever the detector actually records. That’s typically one of:

- *Calorimeter clusters* – energy deposits in electromagnetic/hadronic calorimeters.
- *Charged tracks* – trajectories measured in the inner tracker.
- *Particle flow* – a clever combination that uses tracks for charged particles and calorimeters for neutrals, getting the best of both.

The whole point of understanding jet algorithms is that the *same* algorithm, applied to either truth-level particles or detector objects, should give jets with similar properties. This is what allows us to compare theory predictions (truth-level) to data (detector-level) in an apples-to-apples way.

3 Why We Care: Jets Are Everywhere at the LHC

Before diving into algorithms, let’s motivate all this. Why are jets such a big deal at the LHC?

- **Standard Model tests.** QCD dominates the LHC event rate, and multijet cross sections are a primary way to measure α_s and test QCD at the highest energies.
- **Hadronic decays of heavy particles.** The W boson, the Z boson, the top quark, and the Higgs *all* decay hadronically a large fraction of the time. Reconstructing those decays means

reconstructing jets. For example, $t \rightarrow Wb \rightarrow q\bar{q}b$ gives three jets from a single top quark.

- **New physics searches.** Most BSM models predict final states with jets – heavy particles cascading down through quarks. Searches for supersymmetry, dark matter, and extra dimensions all depend on jets.

The Comparison to Leptons

Leptons (electrons, muons) are “clean”: they don’t hadronize, they leave a single well-defined track, their energy is well measured. Jets are messy by comparison: hundreds of particles, soft contamination from pileup and the underlying event, energy that’s smeared across calorimeter cells. *But* jets are what you get when QCD produces something, and QCD produces almost everything at the LHC. So we have to learn to live with the mess.

4 The Central Question: What Counts as a Jet?

Here’s where we get to the heart of it. Given a bunch of particles or calorimeter deposits in an event, how do we decide which ones belong to the same jet? Look at this event:

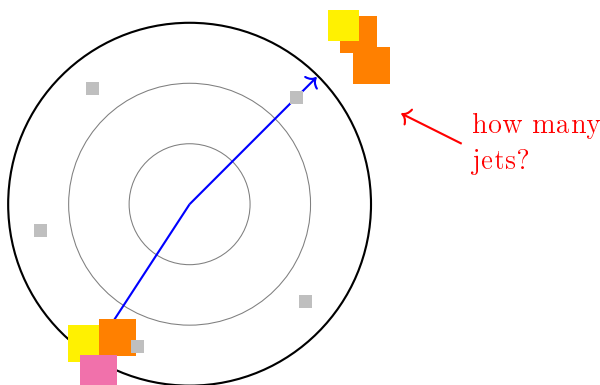


Figure 2: A very simple event. Two clear high-energy clusters plus some scattered low-energy hits. Two jets? Three? Where are the boundaries? This is the question jet algorithms answer.

Now imagine a much busier event – a real high-pileup LHC event where dozens of pp collisions happen in the same bunch crossing and you have thousands of tracks. The question “how many jets?” becomes a genuinely ambiguous one without a precise algorithm.

What We Need

A **jet algorithm** is a precise, deterministic recipe that takes an event (a list of particles/-clusters with 4-momenta) and returns a list of jets (each with a 4-momentum). The same input always gives the same output. Two different algorithms applied to the same event can – and will – give different jets.

5 What Makes a Good Jet Algorithm?

Four properties matter, and together they define what’s called a “good” algorithm:

5.1 Property 1: Parton-jet correspondence

The algorithm should produce jets whose 4-momenta approximate those of the underlying hard partons. If your algorithm systematically misses the jet from the scattered quark, you can't make predictions about it.

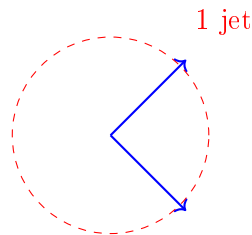
5.2 Property 2: Infrared (IR) safety

This is the most important theoretical property. An algorithm is **infrared safe** if adding a soft particle to the event doesn't change the number or properties of the jets.

Why IR Safety Matters

In QCD, you can radiate a soft gluon from any colored line at any energy – there's no minimum. When you compute a cross section perturbatively, you get *divergences* from these soft emissions. Those divergences cancel between real emission (soft gluon in the final state) and virtual emission (soft gluon in a loop) – this is the Kinoshita-Lee-Nauenberg theorem. For this cancellation to happen in a *jet* cross section, the jet itself has to be defined in a way that's insensitive to soft radiation. If the jet changes when you add a soft gluon, then the real and virtual contributions describe different final states and their divergences don't cancel. **Only IR-safe observables can be calculated in perturbative QCD.**

Without soft emission



With a soft gluon

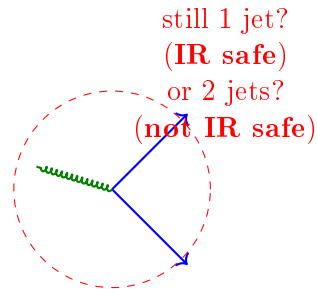


Figure 3: IR safety in a nutshell. If a soft gluon (wiggly line) added to the event changes the number of jets, the algorithm is *not* IR safe and cannot be used with perturbative QCD calculations.

5.3 Property 3: Collinear safety

Similarly, splitting one particle into two collinear particles of the same total momentum should not change the jets.

Single particle

Collinear split



Figure 4: Collinear safety. Splitting a particle into two collinear particles with the same total momentum must leave the jet unchanged. Again, collinear divergences in perturbative QCD require this.

Together, IR safety and collinear safety are often abbreviated **IRC safety**. They are non-negotiable for any modern jet algorithm.

5.4 Property 4: Practical considerations

Beyond the theoretical requirements, we want:

- Detector-independence – the algorithm should work on particle-level and detector-level inputs equally well.
- Computational speed – an LHC event has thousands of particles, and algorithms need to scale well.
- Stability against pileup – in high-luminosity LHC running, multiple pp collisions per bunch crossing pile extra soft activity into every event.

6 Jet Algorithm Family 1: Cone Algorithms

6.1 The iterative cone

The historically first approach: pick the most energetic particle as a seed, draw a cone of radius R around it, compute the jet axis as the sum of all constituents in the cone, re-center the cone on this axis, and iterate until it stops moving. This is the “iterative stable cone” algorithm.

Problem: not IR safe. A soft particle can become the seed for a spurious cone, changing the jets.

6.2 SISConc (Seedless Infrared-Safe Cone)

A cleverer variant: instead of picking seeds, enumerate *all* possible stable cones. This fixes IR safety but scales as $N^2 \ln N$, which is slow for large N .

The Bottom Line on Cones

Cone algorithms are essentially extinct in modern LHC physics. Sequential recombination algorithms (next section) have replaced them because they are faster, more flexible, and have nicer theoretical properties. We mention cones here mainly for historical context and because you’ll see them in older papers.

7 Jet Algorithm Family 2: Sequential Recombination

This is where the real action is. The idea, motivated by a picture of undoing the parton shower, is:

The Clustering Idea

Step 1. For every pair (i, j) of constituents, compute a “distance” d_{ij} . Also compute, for each constituent, a “beam distance” d_{iB} .

Step 2. Find the smallest of all these distances.

- If the smallest is some d_{ij} , merge constituents i and j into a new pseudo-particle (sum their 4-momenta).
- If the smallest is some d_{iB} , declare i a completed jet and remove it from the list.

Step 3. Repeat until no constituents are left.

The outputs are a list of jets.

So what’s the distance? That’s where the three modern algorithms differ.

7.1 The generalized distance formula

The k_T , Cambridge/Aachen, and anti- k_T algorithms *all* share a single master formula, differing only in one exponent:

The Unified Distance Formula

$$d_{ij} = \min(k_{ti}^{2p}, k_{tj}^{2p}) \frac{\Delta_{ij}^2}{R^2}, \quad d_{iB} = k_{ti}^{2p} \quad (1)$$

where:

- k_{ti} is the transverse momentum of constituent i .
- $\Delta_{ij}^2 = (\eta_i - \eta_j)^2 + (\phi_i - \phi_j)^2$ is the angular separation in the rapidity–azimuth plane. (This is the same ΔR you met when rapidity was introduced: $\Delta R = \sqrt{\Delta\eta^2 + \Delta\phi^2}$.)
- R is the *radius parameter* of the jet, typically $R = 0.4$ or $R = 1.0$.
- p is the *algorithm parameter* that picks which algorithm you’re running.

The three canonical choices of p :

p value	Algorithm name	Characteristic behavior
+1	k_T	Clusters hard collinear radiation first
0	Cambridge/Aachen (C/A)	Clusters geometrically closest first
−1	anti- k_T	Clusters around hardest particle; round jets

7.2 Understanding what each algorithm does

This takes a bit of thought – let’s work through it.

k_T algorithm ($p = 1$). Distance is $\min(k_{ti}^2, k_{tj}^2)\Delta_{ij}^2/R^2$. For a low- p_T particle, k_t^2 is small, so distances involving it are small – those are clustered first. But for two *hard* particles with small Δ_{ij} , the distance is also small. So k_T effectively clusters nearest-in-angle hard collinear radiation first. This mimics the *inverse* of a PYTHIA-like parton shower.

Cambridge/Aachen ($p = 0$). Distance is just Δ_{ij}^2/R^2 – purely geometric, no p_T weighting. The algorithm clusters based on angular proximity alone. This mimics the inverse of a HERWIG (angular-ordered) shower.

anti- k_T ($p = -1$). Distance is $\min(1/k_{ti}^2, 1/k_{tj}^2)\Delta_{ij}^2/R^2$. Now *high- p_T* particles have *small* distances to nearby things – so clustering grows outward from the hardest particles. This has no nice parton-shower inverse interpretation, but it produces **round jets** that are stable against soft radiation.

Why Anti- k_T Wins in Practice

The ATLAS and CMS experiments use anti- k_T almost exclusively. Why? Because anti- k_T jets have *conical boundaries* (they look like ideal cones!), their areas are insensitive to pileup, and they cluster from the hard core outward – which is what you’d intuitively want a jet to do. The k_T algorithm, by contrast, clusters from the outside in, and pileup contamination can “grow” its jets in unpredictable ways. Anti- k_T gives you the best of both worlds: the robustness of sequential recombination and the intuitive round shape of a cone.

7.3 A worked picture

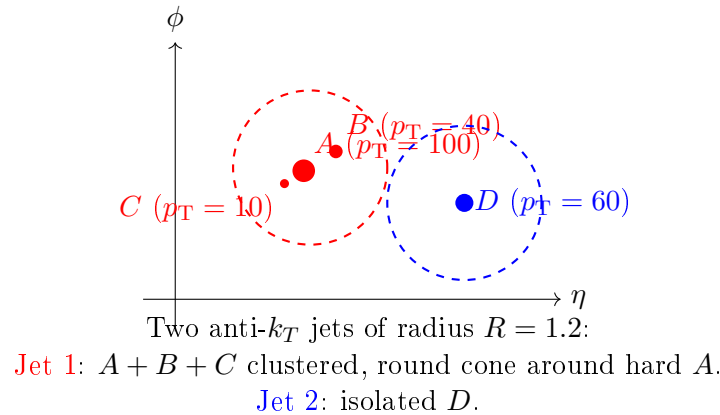


Figure 5: Schematic anti- k_T result for a 4-particle event. The algorithm grows cones outward from the hard particles A and D ; particles inside R are absorbed. This illustrates why anti- k_T jets end up looking like nice round cones.

8 Jets of Heavy Particles: Boosted Jets and Substructure

We’ve covered the “basic” jet-finding problem. But modern LHC physics goes further. When a W , Z , t , or Higgs decays hadronically at high p_T , its decay products get collimated into a single “fat” jet. Can we tell these apart from ordinary QCD jets?

8.1 The boost argument

A heavy particle of mass m decaying into two massless products at $p_T \gg m$ has its decay products separated by roughly:

Angular Separation of Boosted Decays

$$\Delta R \approx \frac{2m}{p_T} \quad (2)$$

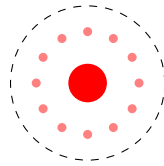
So at $p_T = 500 \text{ GeV}$, a $W \rightarrow q\bar{q}$ decay (with $m_W \approx 80 \text{ GeV}$) gives $\Delta R \approx 0.32$, which fits inside a single jet of radius $R = 1.0$.

This is why people use “fat jets” with $R = 1.0$ for boosted objects: the entire $W/Z/H/t$ decay is inside one jet.

8.2 Looking inside the jet: substructure

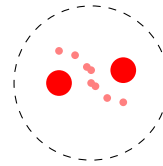
The key insight is that a boosted $W \rightarrow q\bar{q}$ jet looks different *internally* from an ordinary QCD jet:

Ordinary QCD jet



single hard core,
soft stuff around it

Boosted $W \rightarrow q\bar{q}$



two hard cores,
mass peak at m_W

8.3 The BDRS mass-drop tagger

The first practical tool for this was the **BDRS tagger** (Butterworth, Davison, Rubin, Salam – they built it for the Higgs). The idea:

BDRS Algorithm

1. Cluster the event with Cambridge/Aachen (it’s angle-ordered, so the last splitting in the clustering tree is the widest).
2. Undo the clustering history one step at a time. At each step, the C/A tree gives you two subjets that just merged.
3. Check two conditions:
 - *Mass drop*: $\mu = m_{j_1}/m_j$. The heavier subjet should carry most of the mother’s mass (i.e. $\mu < \text{some threshold}$).
 - *Subjet asymmetry*: $y = \min(p_T^1, p_T^2) \cdot (\Delta R_{12})^2 / m_{j_{\text{et}}}^2$ should be *large* (the two subjets should share energy roughly equally, as expected from a real $W \rightarrow q\bar{q}$ decay).
4. If both conditions pass, you’ve found the hard $W \rightarrow q\bar{q}$ splitting. If not, discard the

softer subjet and repeat on the harder one.

The idea is to “peel off” soft radiation until you’re left with the true two-pronged hard structure. Once you’ve identified the hard prongs, you can compute their combined mass and check if it’s near m_W .

8.4 Jet grooming: trimming and softdrop

Closely related: **grooming**. Even after you find a heavy-object jet, soft radiation (from pileup, underlying event, initial-state radiation) smears its mass distribution. Grooming removes this contamination.

Trimming: recluster the jet’s constituents with k_T at a smaller radius R_{sub} , and discard any subjet whose p_T is below a fraction f_{cut} of the original jet’s p_T .

Softdrop: a more theoretically motivated successor based on the splitting pattern of the C/A tree.

9 Tying It All Together: The W Tagger Example

Here’s how all these pieces combine in a real analysis. To tag a jet as coming from a boosted W :

1. Run anti- k_T with $R = 1.0$ to find fat jets.
2. Groom each fat jet (trimming).
3. Check the groomed mass is near $m_W \approx 80 \text{ GeV}$.
4. Cut on a substructure variable (the 2-prong “D2” variable is a popular choice) to reject QCD background.

The payoff is dramatic: you can distinguish W jets from QCD multijet background with good efficiency and rejection, opening up entire classes of physics searches that would otherwise be buried in QCD.

10 Summary of Part 2

Main Takeaways

1. A **jet** is a collimated spray of hadrons that approximates the kinematics of a parent parton. Jets come in truth-level (from MC particles) and detector-level (from calorimeter/tracker) varieties.
2. A **jet algorithm** is a precise recipe for grouping constituents into jets. Different algorithms give different jets.
3. **IRC safety** (infrared + collinear safety) is mandatory for any algorithm used with perturbative QCD calculations. Adding soft radiation or splitting particles collinearly must not change the jets.
4. **Sequential recombination** is the modern standard: $d_{ij} = \min(k_{ti}^{2p}, k_{tj}^{2p}) \Delta_{ij}^2 / R^2$. $p = 1$ is k_T ; $p = 0$ is Cambridge/Aachen; $p = -1$ is **anti- k_T** , which is what ATLAS and

CMS use.

5. **Anti- k_T** produces round, pileup-stable jets by growing cones outward from hard particles.
6. At high p_T , heavy-particle decays collimate into a single jet ($\Delta R \approx 2m/p_T$). **Substructure** variables and **grooming** techniques let us tag these boosted objects and separate them from QCD background.